Concise iPRA research

A uniform introduction rule for intuitionistic primitive recursive arithmetic

An unrestricted seven-rule calculus,
both interpretation composites,
and a measured checker comparison

Prepared for Vladimir Reshetnikov
November 11, 2026

Result and limitation. The new calculus has a single introduction schema whose mode depends only on the matrix root. Every equation, Q formula, and D formula belongs to its public language. A reversed iterator orbit supplies the missing progress elimination, and the report proves bi-interpretability with full first-order iPRA.

This is a structural simplification, not a proof of a globally smallest axiomatization. In matched straightforward implementations it is smaller than the adopted eight-rule kernel, but larger than the already known seven-rule witness-sensitive kernel. A separate, small implementation reduction improves the inspected compact envelope; it is not an implementation of the new orbit calculus.

Reference snapshot. `VladimirReshetnikov/ProveIt`
`a4e5e96c456303cf231eab2aeeb3291d8889833f`
`Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`

Abstract

We replace the separate case and equation-induction introductions of the current native scalar kernel by one introduction. At an equation matrix, $Q_y E$ means $\forall y(E(y) \to E(Sy))$; at a compound matrix it keeps the conditional reading $B(0) \to \forall y B(y)$. The corresponding D binds an existential witness. Elimination specializes a closure along $I(a, n; u.Su)$, with the source placed in the counter position. Its zero source computes directly to the arbitrary target n, which derives contextual equation induction without a circular use of an iterator identity. The new rules admit numerical substitution by direct induction. Explicit translations preserve derivations in both directions and have provably identity formula composites for the entire language, not only guarded public formulas. Numerical domains remain whole and equality remains actual equality. The written theorem is accompanied by a standalone typed proof checker, matched old kernels, reproducible size measurements, and positive and negative certificates. The new metatheorems are not claimed to have been checked in Lean or Rocq.

Contents

1 The comparison target and the meaning of simplicity **2**
1.1 What the size claim is . . . 2

2 A seven-rule calculus with complete progressive elimination **3**
2.1 Complete primitive specification . . . 3
2.2 Equality and the induction bootstrap . . . 4
2.3 Ordinary soundness . . . 6
2.4 A precise comparison with the existing native calculus . . . 6
2.5 Logical macros and the order of the completeness proof . . . 7
2.6 Both formula composites for the entire language . . . 8
2.7 Bi-interpretability, consistency, and the inherited arithmetic . . . 9

3 Complete checkers and measured simplicity **10**
3.1 The main typed checker and matched controls . . . 10
3.2 A separate compact implementation improvement . . . 11
3.3 What the executable evidence checks . . . 12

4 Proof audit and limits of the reduction **12**
4.1 The order of the arithmetic bootstrap . . . 12
4.2 Why equational transport suffices . . . 13
4.3 Root-sensitive closure is not unrestricted congruence . . . 13
4.4 A concrete obstruction to keeping the old atomic reading . . . 14
4.5 What the comparison establishes . . . 14

A Explicit logical macros in the reference language **15**

B The arithmetic interface and the inherited bridge **16**

C Artifacts and reproducibility **17**
C.1 The seven proof-instruction branches . . . 18

1 The comparison target and the meaning of simplicity

The reference is the exact repository snapshot displayed on the title page, inspected through GitHub. Its main native kernel $\mathcal{N}$ has equations, a one-variable closure Q, and a two-variable binder D. It has eight primitive rule constructors: hypothesis, two computation equations, equational transport, Q elimination, D elimination, D introduction by cases, and Q introduction by equation progress. The repository proves its equivalence with full first-order iPRA in Lean [1, 2, 3].

The reference also contains a seven-rule presentation W. Its single introduction uses progress exactly when the matrix is an equation and the witness is the literal zero; it uses cases otherwise. The theorem `wprf_iff_nprf` proves that this presentation has the same sequents as the native kernel. Consequently, a claim of novelty based on the number seven alone would be incorrect. A numerical substitution can change a witness into literal zero, so a direct induction on W derivations does not preserve the primitive introduction choice [4].

The present construction changes the meaning and elimination of atomic closures. This makes the introduction choice depend only on the formula root, which numerical substitution preserves. Unlike the guarded presentations [5], every formula generated by the displayed grammar is public. No restriction to a prescribed zero guard, protected equation, or mirror shape is imposed.

Presentation	Introductions	Primitive mode selection	Public formulas
Adopted $\mathcal{N}$	Cases and equation progress	Two rules	All Eq/Q/D
Known W	One	Matrix root and literal witness	All Eq/Q/D
Guarded core	One	Matrix root	Restricted atomic matrices
New $\mathcal{R}$	One	Matrix root only	All Eq/Q/D

The scalar terms, arithmetic construction, and use of progressive conditions are inherited ideas. In particular, an earlier progressive presentation uses progress at every matrix root, counter-aware recursion, separate step and iteration eliminations, and an explicit Boolean-cases rule [7]. Here the iterator remains scalar, compound closures retain their conditional reading, there is no primitive Boolean-cases rule, and one orbit specialization serves both kinds of closure. The all-formula normalization theorem is the substantive new result.

1.1 What the size claim is

The relevant implementation criterion is the size of a complete standalone proof checker, with gzip level 9 reported separately for information. We measure significant Python tokens, all AST nodes, and UTF-8 bytes after removing genuine docstrings and applying `ast.unparse`, followed by one newline. No producer or theorem database is imported by a checker.

Two comparisons are needed. The primary comparison uses explicit tagged syntax, exact integer/list checks, de Bruijn scoping, and complete Eq/Q/D statement validation in three matched standalone programs. The secondary comparison concerns the repository's much smaller arity codec under its canonical-public-goal convention. That convention does not include a complete public-grammar validator. Its numbers cannot be interchanged with those of the typed interface.

The inspected incoming archives are part of the comparison, not omitted competitors. Their original ZIP bytes were authenticated against the six Git blob identifiers at the pinned commit.

They include `deterministic_auxiliary`, `guarded_source`, `guarded_orbit`, `mirror_guard`, `protected_core`, and `reversal_core` [8]. Exact source records and the comparison measurements are included in the archive.

Scope of the outcome. The new theory gives a cleaner introduction discipline and preserves full bi-interpretability. It does not improve every existing checker-size coordinate. The separate compact implementation gain is small and does not establish a new mathematical lower bound. No global minimality theorem is claimed.

2 A seven-rule calculus with complete progressive elimination

The new presentation keeps the scalar iterator and the three native formula constructors. It changes the meaning of an atomic closure: an atomic closure records a progressive equation, while a compound closure retains the old conditional-universal meaning. A single elimination rule recovers both one-step progress and induction from an atomic closure. Consequently every formula of the new language has a faithful ordinary reading; no restricted public grammar or auxiliary-only formula class is needed.

This is a structural simplification, not a claim of an absolute minimum. The current Lean kernel has eight rule constructors. The proposed kernel has seven, and its introduction rule commutes directly with numerical substitution. The earlier seven-rule `WPrf` presentation also has seven constructors, but its introduction distinguishes the literal witness 0 from other witnesses. Here the distinction depends only on whether a matrix is a literal equation, which numerical substitution cannot change. The concrete checker measurements, including comparisons in which this calculus is larger, are reported separately.[2, 4]

2.1 Complete primitive specification

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u is bound in h; other occurrences are free parameters. Substitution is capture-free. Put

$$1 := S0, \qquad \mathsf{A}(a, n) := I(a, n; u.Su). \tag{1}$$

Thus the first argument of A is the iteration counter and the second is its initializer. This argument order is essential below.

Formulas are

$$F ::= (s = t) \mid Q_y F \mid D_{x,y} F. \tag{2}$$

The binder Q binds y, and D binds the distinct variables x, y. A *literal equation* has outermost constructor $=$. A *compound* formula has outermost constructor Q or D. Define the syntactic selector

$$\eta_B(a) := \begin{cases} a, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{3}$$

Write $\Gamma \vdash_{\mathcal{R}} F$ for derivability. Contexts are finite reusable lists of formulas. The complete primitive inventory is the following seven rule schemas:

$$\frac{F \in \Gamma}{\Gamma \vdash_{\mathcal{R}} F} \ \text{Hyp},$$

$$\frac{}{\Gamma \vdash_{\mathcal{R}} I(0, a; u.h) = a} \ \text{I0}, \qquad \frac{}{\Gamma \vdash_{\mathcal{R}} I(Sn, a; u.h) = h[I(n, a; u.h)/u]} \ \text{IS},$$

$$\frac{\Gamma \vdash_{\mathcal{R}} s = t \qquad \Gamma \vdash_{\mathcal{R}} p(s) = q(s)}{\Gamma \vdash_{\mathcal{R}} p(t) = q(t)} \quad \mathrm{Tr}_{=}, \tag{4}$$

$$\frac{\Gamma \vdash_{\mathcal{R}} Q_y B(y) \qquad \Gamma \vdash_{\mathcal{R}} B(\eta_B(a))}{\Gamma \vdash_{\mathcal{R}} B(\mathsf{A}(\eta_B(a), n))} \quad \mathrm{QE}_{\mathsf{A}}, \tag{5}$$

$$\frac{\Gamma \vdash_{\mathcal{R}} D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash_{\mathcal{R}} F}{\Gamma \vdash_{\mathcal{R}} F} \quad \mathrm{DE}, \tag{6}$$

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash_{\mathcal{R}} B(t, Sz)}{\Gamma \vdash_{\mathcal{R}} D_{x,y} B(x,y)} \quad \mathrm{DI}_{\eta}. \tag{7}$$

In (4), $p(v), q(v)$ are numerical term templates. In (5), a, n are arbitrary terms in the conclusion's context. In DE, b is fresh for Γ, F and all fixed parameters of B. In DI, z is fresh for Γ, t and all fixed parameters of B. The displayed local assumptions are discharged. The designated contradiction is $0 = 1$.

There are no other primitive axioms, inference rules, induction schemas, logical connectives, successor axioms, or numerical case principles. In particular, equality transport in a compound formula is not primitive.

Lemma 2.1 (Structural rules and substitution)**.** *Weakening, exchange, contraction, renaming, assumption cut, and capture-free numerical substitution are admissible in $\mathcal{R}$.*

Proof. Use induction on a finite derivation, renaming eigenparameters before a substitution or a cut. A cut replaces each relevant hypothesis leaf by the supplied derivation, weakened into its local context. The only new substitution check is

$$(\eta_B(a))\sigma = \eta_{B\sigma}(a\sigma).$$

It holds because numerical substitution preserves the outermost formula constructor. The iterator abbreviation (1) also commutes with substitution. Hence QE and DI translate to instances of the same rules; there is no witness-literal exception to repair. □

2.2 Equality and the induction bootstrap

A useful derived rule is

$$\frac{\Gamma, B(\eta_B(z)) \vdash_{\mathcal{R}} B(Sz)}{\Gamma \vdash_{\mathcal{R}} Q_y B(y)} \quad \mathrm{QI}_{\eta}. \tag{8}$$

To obtain it, apply DI to $D_{x,y}B(y)$ with x unused and witness 0. Apply DE to this result; its local hypothesis is precisely $Q_y B(y)$, which can be exported by Hyp because it is independent of the fresh witness.

Lemma 2.2 (Numerical equality and contextual equation induction)**.** *The usual reflexivity, symmetry, transitivity, and numerical congruence rules are derivable. For every literal equation $E(y)$,*

$$\frac{\Gamma \vdash_{\mathcal{R}} E(0) \qquad \Gamma, E(z) \vdash_{\mathcal{R}} E(Sz)}{\Gamma \vdash_{\mathcal{R}} E(t)} \quad (z \textit{ fresh}) \tag{9}$$

is admissible with arbitrary retained side assumptions Γ.

Proof. Let $J(a) = I(0, a; u.0)$. I0 gives $J(a) = a$; transport this equation along itself with template $v = a$ to obtain $a = a$. The usual equational templates then give symmetry, transitivity, and numerical congruence.

For induction, the step premise and (8) give $Q_y E(y)$. Use (5) with source $a = 0$ and initializer $n = t$. Its conclusion is $E(I(0, t; u.Su))$. I0 and purely equational transport give $E(t)$. No

identity $I(t, 0; u.Su) = t$ has been assumed. Using the counter t and initializer 0 in the elimination rule instead would not justify this bootstrap. □

Lemma 2.3 (Elementary iterator identities)**.** *The new calculus proves, with arbitrary parameters,*

$$\begin{aligned} &\mathsf{A}(0,n) = n, && \mathsf{A}(Sa,n) = S\mathsf{A}(a,n), \\ &\mathsf{A}(a,0) = a, && \mathsf{A}(a,Sn) = S\mathsf{A}(a,n), \\ &\mathsf{A}(a,n) = \mathsf{A}(n,a), && \mathsf{A}(a,1) = Sa. \end{aligned} \tag{10}$$

Proof. The first row consists of computation instances. Induct on a for $\mathsf{A}(a,0) = a$. Induct on a again for the initializer-successor identity: its base is $Sn = Sn$, and its step follows by applying S to the induction hypothesis. For commutativity, induct on a. The base is $n = \mathsf{A}(n,0)$; the step is

$$\mathsf{A}(Sa,n) = S\mathsf{A}(a,n) = S\mathsf{A}(n,a) = \mathsf{A}(n,Sa).$$

Finally $\mathsf{A}(a,1) = S\mathsf{A}(a,0) = Sa$. All inductions are instances of (9) on literal equations. □

An atomic closure therefore has the derived one-step elimination

$$Q_y E(y),\ E(a)\ \vdash_{\mathcal{R}}\ E(Sa). \tag{11}$$

Take initializer 1 in QE and use (10). For a compound matrix, QE at source 0 and initializer t has the ordinary conditional-closure effect $Q_y B, B(0) \vdash B(t)$, once transport through B has been derived.

Lemma 2.4 (Full transport and homogeneous binder congruence)**.** *For every formula template $F(v)$,*

$$s = t,\ F(s)\ \vdash_{\mathcal{R}}\ F(t). \tag{12}$$

Moreover, if B and C are uniformly interderivable and have the same root class (both equations or both compound), their Q closures are interderivable, and their D closures are interderivable.

Proof. For transport, induct on F. Equations use the primitive rule. For $Q_y E(v,y)$ with atomic E, introduce the target closure by QI. Reverse-transport its local $E(t,z)$ to $E(s,z)$, use (11) for the source closure, and transport the resulting $E(s,Sz)$ to $E(t,Sz)$.

For $Q_y B(v,y)$ with compound B, introduce the target closure from $B(t,0)$. The induction hypothesis converts this to $B(s,0)$. QE for the source closure at initializer Sz gives $B(s, I(0,Sz;u.Su))$. The induction hypothesis applied inside the strictly smaller matrix B removes the I0 redex and changes the parameter s to t. This proves the QI target $B(t,Sz)$.

For $D_{x,y}B(v,x,y)$, eliminate the source binder at a fresh witness b. Introduce the target at that witness. Convert its local assumption $B(t,b,\eta_B(z))$ to $B(s,b,\eta_B(z))$ by the induction hypothesis. At an atomic matrix use (11); at a compound matrix use QE with source 0, initializer Sz, and I0. Transport the successor result back to the parameter t. Close DI and DE. Every invoked formula transport concerns the strictly smaller matrix, so this is not circular.

For homogeneous congruence, use the same constructions, replacing numerical transport by the two given conversions between B and C. Their root classes agree, so their local DI assumptions use the same value of η. For D, eliminate the source at a fresh witness and introduce the target at that witness. The argument works uniformly under arbitrary additional retained assumptions. □

The root-class qualification is substantive. The new Q is not claimed to preserve arbitrary equivalence when one matrix is an equation and the other is compound. A progressive assertion and a conditional-universal assertion are different assertions even when their matrices are pointwise equivalent. The translations below respect this boundary explicitly.

2.3 Ordinary soundness

Use the pinned repository's scalar theory T_{eq}: ordinary intuitionistic first-order logic with actual equality, the two iterator computations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = 1) \to \bot$, and contextual induction for literal equations. The repository proves its whole-domain bi-interpretability with iPRA.[3]

Define the new ordinary reading recursively by

$$\begin{aligned} [\![s = t]\!] &= (s = t), \\ [\![Q_y E(y)]\!] &= \forall y\,(E(y) \to E(Sy)) \quad (E \text{ a literal equation}), \\ [\![Q_y B(y)]\!] &= [\![B(0)]\!] \to \forall y\,[\![B(y)]\!] \quad (B \text{ compound}), \\ [\![D_{x,y} B]\!] &= \exists x\,[\![Q_y B]\!]. \end{aligned} \tag{13}$$

This translation preserves all free numerical variables and commutes with numerical substitution. It is a finite syntactic translation, not a truth predicate on numerical formula codes.

Theorem 2.5 (Soundness)**.** *If* $\Gamma \vdash_{\mathcal{R}} F$*, then* $[\![\Gamma]\!] \vdash_{T_{\mathrm{eq}}} [\![F]\!]$.

Proof. Hyp, computations, equational transport, and DE have their ordinary meanings. Atomic DI is universal introduction of progress, followed by existential introduction at its witness. Compound DI is conditional universal introduction using numerical coverage, followed by existential introduction.

For atomic QE, retain $H = \forall y(E(y) \to E(Sy))$ and $E(a)$. Equation induction on n gives $E(\mathsf{A}(n, a))$: the base is $E(a)$, and the step uses H at $\mathsf{A}(n, a)$ and IS. The pure equational proofs in Lemma 2.3 also hold in T_{eq}, so commutativity gives $E(\mathsf{A}(a, n))$. For compound QE the minor is $[\![B(0)]\!]$; its conditional-universal major directly supplies the target instance. Thus every rule translates soundly. □

2.4 A precise comparison with the existing native calculus

Let $\mathcal{N}$ denote exactly the current native Eq/Q/D calculus `NPrf`. Its $Q_y B$ always means $B(0) \to \forall y B(y)$; its direct progress rule introduces $Q_y E$ only for equations. Its terms and numerical equality are identical to those of $\mathcal{R}$.

For $E \equiv (s = t)$ define

$$\begin{aligned} C(v, a, b) &:= I(v, a; u.b), \\ G_v E &:= (C(v, 0, s) = C(v, 0, t)), \\ L(E) &:= D_{r,v} G_v E, \\ P(F) &:= \begin{cases} L(F), & F \text{ an equation}, \\ F, & F \text{ compound}. \end{cases} \end{aligned} \tag{14}$$

All newly displayed binders are fresh; the update in C ignores u, and r is unused in $L(E)$.

Lemma 2.6 (Padding)**.** *In $\mathcal{R}$, $L(E)$ and E are interderivable under arbitrary contexts. Consequently $P(F)$ is compound and interderivable with F.*

Proof. Computations and equality prove G_0E and $G_{Sz}E \dashv\vdash E$. From E, introduce $L(E)$ by atomic DI, ignoring its local induction assumption and using E for the successor target. Conversely, DE on $L(E)$ gives Q_vG_vE. Supply the theorem G_0E and apply QE with source 0, initializer 1. I0 gives G_1E, which reduces to E. This proof uses no ordinary logical macro. □

Define a translation $T : \mathcal{N} \to \mathcal{R}$ by

$$\begin{aligned} T(s = t) &= (s = t), \\ T(Q_yB) &= Q_yP(TB), \\ T(D_{x,y}B) &= D_{x,y}P(TB). \end{aligned} \tag{15}$$

It fixes terms, commutes with numerical substitution, and places a compound matrix immediately below every translated binder.

Theorem 2.7 (Translation into the new calculus)**.** *Every $\mathcal{N}$ derivation translates effectively:*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longrightarrow \quad T\Gamma \vdash_{\mathcal{R}} TA.$$

Proof. Induct on the old derivation. Hyp, computations, and equational transport translate literally. Old QE becomes compound new QE: pad the minor, specialize at the old target using source 0, remove the I0 redex, and unpad. In DE the opened new assumption is exactly $Q_yP(TB)(b, y)$, the translation of the old opened Q assumption. For case DI, unpad the local base, apply the translated premise, and pad the successor conclusion; the new matrix is compound.

For old direct Q-progress at E, the translated target is $Q_yL(E(y))$. Introduce it by compound QI. Its local base $L(E(0))$ supplies $E(0)$. Apply the recovered contextual equation induction to the translated step, then pad $E(Sz)$ to $L(E(Sz))$. This discharges the only nonliteral rule translation. Each construction is finite and preserves eigenvariable conditions after fresh renaming. □

2.5 Logical macros and the order of the completeness proof

The old ordinary logical macros will be denoted $\bot_{\mathcal{N}}$, $\to_{\mathcal{N}}$, $\wedge_{\mathcal{N}}$, $\vee_{\mathcal{N}}$, $\exists_{\mathcal{N}}$, and $\forall_{\mathcal{N}}$. For specificity they are the macros of the pinned `NativeLogic.lean`:[2]

$$\begin{aligned} \bot_{\mathcal{N}} &= (0 = 1), \\ A \to_{\mathcal{N}} B &= Q_y \operatorname{sel}(y, A, B), \\ A \wedge_{\mathcal{N}} B &= Q_y \operatorname{gate}(y, \operatorname{sel}(\operatorname{par}(y), A, B)), \\ \exists_{\mathcal{N}} x\, A(x) &= D_{x,y} \operatorname{gate}(y, A(x)), \\ A \vee_{\mathcal{N}} B &= \exists_{\mathcal{N}} v\ \operatorname{sel}(v, A, B), \\ \forall_{\mathcal{N}} x\, A(x) &= A(0) \wedge_{\mathcal{N}} Q_x A(x). \end{aligned} \tag{16}$$

Here $\operatorname{par}(y) = I(y, 0; u.C(u, 1, 0))$. The old gate acts on equations as G_v in (14). The exact finite recursive definitions of the gate and selector, including their behavior under Q and D, are stated in Appendix A. The selector has zero endpoint A and successor endpoint B. All these operations concern the *old* uniform conditional calculus. The pinned repository supplies their capture-free definitions and derived rules.

Lemma 2.8 (Logic on image formulas, with arbitrary side context)**.** *For old formulas A, B, the formulas $T(A \to_{\mathcal{N}} B)$, $T(A \wedge_{\mathcal{N}} B)$, and the analogous translated macros have their ordinary natural-deduction rules on principal formulas TA, TB. These derived rules permit an arbitrary retained $\mathcal{R}$ context and arbitrary supplied $\mathcal{R}$ premise proofs. Image existential elimination also permits an arbitrary new conclusion, not necessarily an image formula.*

Proof. Translate the finite *derived-rule constructions*, rather than merely closed old theorems. Their nonlogical steps are replaced by the local simulations in Theorem 2.7. Their open premise leaves are filled with the supplied new proofs; assumption cut and weakening (Lemma 2.1) retain any additional new context. The construction never examines the syntax of passive side assumptions. Thus this is a uniform rule simulation, not an inference from closed-theorem preservation alone.

For the important existential exception, the old elimination construction opens its defining D by DE, obtains the witness instance from the opened gated closure, and cuts this instance into the supplied minor derivation. It neither analyzes nor introduces the minor's conclusion. The translated construction consequently works for every new conclusion F, using the same freshness condition. This is precisely the structure of the pinned `NPrf.exE` proof. □

We do *not* yet assert that the old selectors or logical macros can be applied unchanged to arbitrary mixed formulas. Such an assertion would incorrectly ignore the root distinction in Q. The next normalization argument first proves that every mixed formula is equivalent to an image.

2.6 Both formula composites for the entire language

Define $J : \mathcal{R} \to \mathcal{N}$, fixing all terms and equations, by

$$\begin{aligned} J(Q_y E) &= \forall_{\mathcal{N}} y\,(E(y) \to_{\mathcal{N}} E(Sy)), && E \text{ an equation},\\ J(Q_y B) &= Q_y JB, && B \text{ compound},\\ J(D_{x,y} E) &= \exists_{\mathcal{N}} x\, \forall_{\mathcal{N}} y\,(E(x,y) \to_{\mathcal{N}} E(x,Sy)), && E \text{ an equation},\\ J(D_{x,y} B) &= D_{x,y} JB, && B \text{ compound}. \end{aligned} \tag{17}$$

This is a fixed finite syntax transformation. It maps compound formulas to compound formulas, preserves free variables, and commutes with numerical substitution. Its ordinary reading in T_{eq} is provably equivalent to (13), by the established meanings of the old logical macros.

Lemma 2.9 (Atomic closure normalization)**.** *For every equation $E(x, y)$, under arbitrary retained contexts,*

$$\begin{aligned} Q_y E(x,y) &\dashv\vdash_{\mathcal{R}} T(\forall_{\mathcal{N}} y(E(x,y) \to_{\mathcal{N}} E(x,Sy))),\\ D_{x,y} E(x,y) &\dashv\vdash_{\mathcal{R}} T(\exists_{\mathcal{N}} x\, \forall_{\mathcal{N}} y(E(x,y) \to_{\mathcal{N}} E(x,Sy))). \end{aligned} \tag{18}$$

Proof. Write $H(x)$ for the image formula on the first line's right. From $Q_y E(x, y)$ and the temporary assumption $E(x, z)$, derived one-step elimination gives $E(x, Sz)$. The image implication and universal introductions of Lemma 2.8 give $H(x)$. Their principal component formulas are just equations, which are already fixed by T; no full mixed-language logical rule has been assumed. Conversely, under $H(x)$ and $E(x, z)$, image universal specialization and implication elimination give $E(x, Sz)$; atomic QI gives $Q_y E(x, y)$.

For D, eliminate the left-hand binder at fresh b, normalize its local $Q_y E(b, y)$ to $H(b)$ by the first part, introduce the image existential at b, and close DE. In the reverse direction, image existential elimination supplies a fresh b and $H(b)$. Under $E(b, z)$ its image universal and

implication rules give $E(b, Sz)$, so atomic DI at witness b gives $D_{x,y}E$. Close image existential elimination. Its conclusion is allowed to be this arbitrary new formula by the last assertion of Lemma 2.8. □

Theorem 2.10 (Exact native-language round trips)**.** *For every old formula A and every new formula F,*

$$A \dashv\vdash_{\mathcal{N}} JT(A), \qquad F \dashv\vdash_{\mathcal{R}} TJ(F). \tag{19}$$

These equivalences are uniform under arbitrary additional retained contexts.

Proof. For the old-side composite, note first that $JL(E)$ is old-equivalent to E. It asserts existence of an unused witness and progressivity of the guarded equation G_vE. From E, every positive guard instance follows, so the progress implications hold. Conversely G_0E is a theorem and progress at 0 yields G_1E, hence E. This uses ordinary logical macros in $\mathcal{N}$, whose correctness is already established. Now induct on the old formula. Every binder created by T has a compound matrix, so J gives its old conditional interpretation. The preceding padding equivalence and the induction hypothesis apply under the old Q and D congruence rules.

For the new-side composite, induct on F. Equations are unchanged. Atomic Q and D cases are exactly Lemma 2.9. Suppose the matrix B is compound. The induction hypothesis gives $B \dashv\vdash TJB$. Both B and TJB are compound, since J and T preserve compound roots. Homogeneous congruence (Lemma 2.4) therefore gives

$$QB \dashv\vdash Q(TJB), \qquad DB \dashv\vdash D(TJB).$$

Because TJB is compound, no padding is inserted on these right sides; they are exactly the required TJ composites. There is no appeal to congruence across unlike root classes. The induction proves normalization for every formula before ordinary logic is extended to arbitrary new formulas, avoiding a circular completeness argument. □

All ordinary logical operations in the new language. Now, and only now, define for arbitrary new formulas

$$\begin{aligned} \bot_{\mathcal{R}} &= (0 = 1), \\ F \diamond_{\mathcal{R}} G &:= T(JF \diamond_{\mathcal{N}} JG) \quad (\diamond \in \{\to, \wedge, \vee\}), \\ \exists_{\mathcal{R}} x\, F &:= T(\exists_{\mathcal{N}} x\, JF), \\ \forall_{\mathcal{R}} x\, F &:= T(\forall_{\mathcal{N}} x\, JF). \end{aligned} \tag{20}$$

The round trips convert arbitrary new principal formulas to their images; Lemma 2.8 then supplies the ordinary rules, and the round trips convert their conclusions back. For disjunction or existential elimination with a new conclusion, normalize that conclusion first if necessary. All eigenvariable conditions are preserved because J, T preserve free numerical variables.

This gives intuitionistic natural deduction for the entire new language, including explosion, ordinary numerical coverage, and the quantified rules. It adds no primitive logical axioms. In particular, the system does not use excluded middle for arbitrary quantified formulas.

2.7 Bi-interpretability, consistency, and the inherited arithmetic

Theorem 2.11 (Full consequence equivalence and bi-interpretability)**.** *The translations T, J preserve derivations in both directions and have the round trips* (19)*. The ordinary realization of $\mathcal{R}$ is equiconsistent and bi-interpretable with full first-order iPRA. Both numerical domains*

are whole, numerical equality is actual equality, and the numerical composite isomorphisms have graph $x = y$.

Proof. Preservation by T is Theorem 2.7. For J, translate a new derivation into T_{eq} using Theorem 2.5; its formulas are provably equivalent to the ordinary readings of their J translations. The pinned native completeness and native formula round-trip theorems translate this ordinary derivation back into $\mathcal{N}$ and identify each resulting formula with its J translation. Equivalently, one may translate each new rule directly using the old ordinary macros and equation induction; the only added arithmetic calculation is the QE soundness proof above. The two exact formula composites are Theorem 2.10.

The remaining scalar-to-iPRA arithmetic is inherited, not reproved by the new elimination rule. The repository's chain constructs a scalar predecessor, a term reflecting actual equality, quantifier-free induction, pairing, and a compiler for each fixed primitive-recursive description. The reverse interpretation expresses a fixed scalar iterator by primitive recursion. Equation induction proves the two numerical compilation round trips. These are the contents of the pinned `biInterp`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, and `pra_native_roundTrip` results.[2, 3]

Composing that existing bi-interpretation with T, J proves the assertion. Our two translations leave every scalar term and the numerical domain unchanged, so they add neither a cut nor a quotient to the existing identity graphs. The designated contradiction $0 = 1$ is fixed. Effective finite proof translations therefore give equiconsistency. This is an external equivalence theorem, not an internal assertion of iPRA's own consistency. □

What changes compared with the earlier progressive proposal. The repository already contains a presentation with a progressive binder on every matrix, separate one-step and equation-iteration eliminations, counter-aware primitive recursion, and an explicit Boolean-cases rule.[7] The progressive reading itself is therefore inherited motivation. The present contribution is the root-dependent complete reading with the scalar iterator unchanged, the single reversed-orbit specialization (5), the removal of a separate case principle, and both round trips for the entire Eq/Q/D formula language. The mixed reading without the stronger elimination would lose information at atomic closures; its soundness alone would not establish the theorem above.

Verification status. The new metatheorems in this section are written proofs. They are not claimed to have been checked in Lean or Rocq. The existing Lean theorem names identify the precise inherited bridge. Executable certificates and checker tests support concrete implementations; they do not replace the universal proof or establish a global lower bound on presentation size.

3 Complete checkers and measured simplicity

3.1 The main typed checker and matched controls

The recommended implementation of $\mathcal{R}$ is `code/orbit_checker.py`. It reads exactly a JSON object with `goal` and `proof` fields, verifies that the goal is a closed formula, synthesizes the proof's conclusion, and requires exact agreement. Term and formula tags, arities, integer indices, binder scopes, context membership, and eigenvariable conditions are checked. A successful run exits normally and prints exactly `ACCEPT` followed by a newline. Any exception or resource failure rejects the input.

The two comparison modules implement the pinned eight-rule $\mathcal{N}$ and seven-rule W calculi with the same syntax, substitution operations, validation, and file interface. They are newly written matched controls; they are not represented as Python files published upstream. All helpers and the JSON interface are included in each standalone module.

Complete typed checker	Tokens	AST	Bytes	Gzip 9
Adopted $\mathcal{N}$, matched	1,235	1,407	4,692	1,199
New orbit $\mathcal{R}$	1,206	1,366	4,553	1,211
Known W, matched	1,173	1,334	4,456	1,198

These numbers use CPython 3.12.14 and zlib 1.3.2. Genuine docstrings are removed from the parsed AST before `ast.unparse` normalization; comments and formatting are consequently absent. Bytes include the final newline. Gzip uses compression level 9 with `mtime=0` and includes its wrapper. The evidence also records raw source bytes, raw gzip sizes, and both source hashes. Gzip is informational and is not minimized.

Relative to the adopted eight-rule control, the new checker saves 139 normalized bytes (about 2.96%), 41 AST nodes, and 29 significant tokens. It nevertheless costs 97 bytes, 32 AST nodes, and 33 tokens more than the matched W checker. Its normalized gzip is also larger than both controls. The uniform introduction and direct substitution theorem therefore do not establish a new minimum under the user's implementation metric.

3.2 A separate compact implementation improvement

The archive also contains three compact implementations of the *earlier guarded-source calculus*. They are not implementations of the new orbit calculus. Their comparison contract is the same as that of the authenticated incoming guarded-source package: canonical guarded public statements, with broader administrative proof data. The compact programs do not themselves reject every noncanonical public statement. The main typed checker, in contrast, validates its complete Eq/Q/D public grammar. These are different input contracts.

Compact complete module	Tokens	AST	Bytes	Gzip 9
Previous token profile	257	285	568	338
New token profile	**256**	283	593	346
Previous AST profile	261	279	603	345
New AST profile	260	**278**	602	344
Previous byte profile	264	301	522	328
New byte profile	263	300	**521**	327

Thus the inspected attained coordinate envelope changes from

$$(257 \text{ tokens}, 279 \text{ AST nodes}, 522 \text{ bytes}) \quad \text{to} \quad (256, 278, 521).$$

Each triple is attained by three different programs. No single module attains all three coordinates. The token profile actually gets longer in bytes and gzip, which is why reporting only the coordinate envelope would conceal relevant information.

The byte and AST changes replace the initial administrative truth `[]` by raw integer `0`, and change neutral major fields in emitted certificates accordingly. These are both spellings of a true administrative resource, not new numerical axioms. At nonneutral major premises the original rule tests are unchanged. At a neutral major, both representations select the same computation or introduction branch and contribute no mathematical assumption. The

old canonical proof-production construction therefore compiles with the new neutral spelling, while soundness uses the same interpretation of retained resources. No complete equivalence claim is made about every malformed or noncanonical raw byte stream.

The token profile instead turns the recursive continuation into a `while` loop and uses an immutable tuple context. Its recursive major-premise call is retained. The loop state is exactly the old continuation state: the current goal, retained context, and global fresh index. Induction on consumed packets identifies the two executions. The global certificate convention is unchanged.

These operations compose techniques already present in the incoming packages. They do not constitute a new arithmetic basis or a new logical axiomatization. Their strict implementation gain is real but very small. The frozen predecessors, their hashes, the changed certificates, and the entire new programs are supplied for inspection.

3.3 What the executable evidence checks

The recorded reproduction run has 270 process expectations: 122 expected acceptances and 148 expected rejections, each checked in ordinary or optimized Python as specified in the manifest. Two additional tests exercise open sequents with displayed assumptions, and four controls exercise capture-avoiding substitution. There are zero mismatches.

Positive examples cover both computation equations, reflexivity derived from computation and transport, equation induction whose hypothesis is actually used, zero- and nonzero-source orbit specializations, compound case introduction, and closure extraction through an unused witness. In particular, a complete closed certificate proves $I(3, 0; u.Su) = 3$ using the new induction construction.

Negative examples include a false equation, wrong computation endpoints, an incorrect orbit antecedent, a future witness, an escaping existential eigenparameter, unbound iterator state, negative hypothesis indices, noninteger variable indices, attempted compound induction, attempted primitive compound transport, malformed instructions, and an open public goal. These address the rules that could otherwise strengthen the theory or break scoping.

One open test derives $0 = 1$ from a *false progressive parity assumption*. It shows that the new closure retains its progressive meaning; it is not a closed contradiction proof. Another attempts to use a compound closure progressively and is rejected. The full assumption lists and their scope are recorded in `tests/open_sequents.json`.

Finite checks support the implementation, while the written metatheorems establish the universal mathematical claim. No number of these finite controls would by itself prove soundness, completeness, bi-interpretability, or a minimality theorem.

4 Proof audit and limits of the reduction

The seven-rule presentation changes the meaning of an atomic closure. Consequently its equivalence with the native kernel needs two formula translations. It is not an identity-language deletion of one native rule. This section records the dependency checks that matter for that claim.

4.1 The order of the arithmetic bootstrap

Put $A(a, n) = I(a, n; u.Su)$. The counter is the first argument. The following order avoids using a derived rule in its own proof.

1. Computation and equational transport give numerical reflexivity, symmetry, transitivity, and term congruence. For example, if $W(t) = I(0, t; u.0)$, the equation $W(t) = t$ is a computation; transporting that equation along itself yields $t = t$.
2. A vacuous witness in DI, followed by DE, gives root-directed Q-introduction. This construction uses no numerical induction.
3. Given an equation-induction step and a base proof $E(0)$, Q-introduction gives the progressive closure of E. Orbit elimination with source 0 and initializer t gives $E(A(0, t))$. The computation $A(0, t) = t$ yields $E(t)$.
4. Only after this recovery of contextual equation induction do we prove $A(a, 1) = Sa$. Orbit elimination with source a and initializer 1 then gives the immediate progress rule $\mathsf{Q}_y E(y), E(a) \vdash E(Sa)$.
5. Full formula transport is proved by structural induction, using this immediate progress rule at atomic closure matrices and the ordinary base-to-instance elimination at compound matrices.

In particular, the proof does not begin by assuming $I(t, 0; u.Su) = t$. That identity is a consequence of the recovered induction. Replacing the target $I(a, n; u.Su)$ in the new elimination rule by $I(n, a; u.Su)$ would destroy the direct computation in the third item. The intended rule and the executable representation must retain the same argument order.

The addition commutation law used to establish soundness in the ordinary scalar theory is harmless at that stage: the target scalar theory already has contextual equation induction. That soundness argument is separate from the internal recovery of induction in the new calculus.

4.2 Why equational transport suffices

For a transport template headed by Q, the induction hypothesis applies only to its strictly smaller matrix. Reverse the supplied numerical equality to convert the target introduction hypothesis to the source hypothesis. Apply the source closure, and convert the resulting successor instance forwards. The atomic case uses the immediate progress rule established above; the compound case uses a zero source in orbit elimination. Root-directed Q-introduction closes the argument.

For a template headed by D, first eliminate the source D at a fresh witness. Introduce the target D at that same witness, performing the preceding conversion inside its matrix. Both eigenparameters are chosen outside the transport equality, the retained context, and the fixed matrix parameters. Every invocation of formula transport in these arguments concerns the smaller matrix. Thus this proof does not silently replace the primitive equational transport rule by a stronger logical axiom.

The same root invariance is useful for substitution: substituting numerical terms never turns a literal equation into a compound formula. The new introduction and elimination modes therefore survive numerical substitution without a special witness-zero case. Renaming eigenparameters first gives the usual substitution proof by induction on derivations.

4.3 Root-sensitive closure is not unrestricted congruence

Padding gives an equation E and a compound formula $P(E)$ that are interderivable. Nevertheless one must not infer

$$\mathsf{Q}_y E(y) \;\dashv\!\vdash\; \mathsf{Q}_y P(E(y))$$

merely from that fact. The first closure is progressive and the second is conditional. Numerical transport is legitimate because it preserves the matrix root. The binder-congruence lemma used in the formula composites requires the same mode on its two sides.

The new-side round trip can be justified without assuming logical completeness for all new formulas in advance. First translate the old logical rules for formulas in the image of the old-to-new translation, allowing an arbitrary passive new context. Atomic new closures normalize to image formulas expressing $\forall y(E(y)\to E(Sy))$, since $E(y)$ and $E(Sy)$ are already image formulas. Atomic witnessed closures then normalize using the image existential. Compound cases use the structural induction hypothesis and same-mode binder congruence. The translations of compound formulas remain compound, so that last use of congruence is valid.

One detail is necessary in the atomic witnessed case: elimination of an image existential must permit an arbitrary new conclusion, not only a conclusion already known to be an image formula. Its translated native macro elimination has precisely this property. It opens a D resource, removes padding and the gate from the witness assumption, and uses the supplied minor derivation. The conclusion is only carried through DE; it is not inspected or introduced beneath a mode-sensitive binder. This prevents a circular appeal to the desired normalization theorem.

4.4 A concrete obstruction to keeping the old atomic reading

The change of reading is essential. A short counterexample uses the primitive recursive parity term

$$p(n) = I(n, 0; u.I(u, 1; v.0)), \qquad E(y) \equiv p(y) = 1.$$

Direct computations give $p(0) = 0$, $p(1) = 1$, and $p(2) = 0$ in the standard natural numbers. Thus

$$E(0) \to \forall y E(y) \quad \text{is true, whereas} \quad \forall y(E(y) \to E(Sy)) \quad \text{is false.}$$

Indeed, the old conditional antecedent is false, while the proposed progress condition fails at $y = 1$.

The new orbit elimination would therefore be unsound if its atomic major kept the old conditional meaning: take source 1 and initializer 1. Its minor $E(1)$ holds and its target is $E(I(1, 1; u.Su)) = E(2)$, which fails. Likewise, merely exporting an ordinary conditional closure from a progressive witnessed formula discards information needed for a full formula round trip. The stronger atomic elimination and the changed atomic reading must be adopted together.

4.5 What the comparison establishes

The new calculus has unrestricted native formula syntax, a single root-directed introduction, and a direct substitution argument. These are structural improvements over the guarded and witness-sensitive presentations. They do not by themselves prove a smaller standalone Python checker. The orbit elimination has a richer numerical target, and a faithful implementation must count that cost. The report's checker measurements must therefore distinguish this calculus from any separate compact implementation of an earlier equivalent presentation.

The arguments above are written metatheoretic proofs. Finite executable certificates check particular derivations and implementation behavior; they are not a new Lean or Rocq verification of either formula interpretation. No absolute minimality theorem follows from the seven-rule count or the measured source sizes.

A Explicit logical macros in the reference language

This appendix fixes the finite macros used by the interpretations. All operators in this appendix belong to the old conditional language $\mathcal{N}$. They must be expanded there before applying the translation T. Applying these definitions unchanged to arbitrary new mixed closures would be incorrect. These definitions reproduce the interface of `NativeGate.lean` and `NativeLogic.lean` [2].

Write $C(v,a,b) = I(v,a;u.b)$, where u is unused, and $1 = S0$. Bindings below are freshened so that no displayed passive parameter is captured. Define an equation–formula selector $U_v(s,t;B)$ by

$$\begin{aligned} U_v(s,t;r=w) &= (C(v,s,r) = C(v,t,w)),\\ U_v(s,t;Q_yB) &= Q_yU_v(C(y,0,s),C(y,0,t);B),\\ U_v(s,t;D_{x,y}B) &= D_{x,y}U_v(C(y,0,s),C(y,0,t);B). \end{aligned}$$

Its two endpoints are the equation $s = t$ and the formula B. Put $G_vA = U_v(0,0;A)$. Then G_0A is provable and $G_{Sz}A \dashv\vdash A$. A selector between arbitrary formulas is

$$\begin{aligned} M_v(s=t,B) &= U_v(s,t;B),\\ M_v(Q_yA,B) &= Q_yM_v(A,G_yB),\\ M_v(D_{x,y}A,B) &= D_{x,y}M_v(A,G_yB). \end{aligned}$$

This recursion terminates on the depth of its first argument; the separate recursion defining U terminates on its formula argument. For every A, B, the old calculus derives $M_0(A,B) \dashv\vdash A$ and $M_{Sz}(A,B) \dashv\vdash B$.

Let $p(y) = I(y,0;u.C(u,1,0))$. The logical operations are

$$\begin{aligned} \bot_* &= (0=1),\\ A \Rightarrow_* B &= Q_yM_y(A,B),\\ A \wedge_* B &= Q_yG_yM_{p(y)}(A,B),\\ \exists_* x\, A(x) &= D_{x,y}G_yA(x),\\ A \vee_* B &= \exists_* v\, M_v(A,B),\\ \forall_* x\, A(x) &= A(0) \wedge_* Q_xA(x). \end{aligned}$$

All new bound variables are fresh. These are definitions by finite syntax, not additional primitive logical axioms or rules.

Why they have the logical rules. Implication introduction uses the zero selector as its local hypothesis and the successor selector as its conclusion; elimination specializes at 1. Existential introduction chooses the witness and uses the successor gate. Existential elimination uses D elimination, the zero-gate theorem, and specialization at 1; its final conclusion can be any formula, independently of the existential matrix.

For conjunction, the gate supplies the base theorem and the computations $p(1) = 1$, $p(2) = 0$ recover the two conjuncts. Numerical cases establish the selector at an arbitrary control from proofs of both alternatives. Disjunction uses witnesses 0 and 1; its elimination uses the derived case rule on an implication to the desired conclusion. The universal macro has its zero instance as one conjunct and every instance as the other conjunct's closure. Finally, $0 = 1$ and the selector computations give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary terms; structural induction on formulas gives falsity elimination.

The full proofs and their substitution lemmas are preserved in the pinned source files included with the package. The new theory's normalization proof uses the translated versions of these

derived-rule constructions on T-image principal formulas before it defines the ordinary logical operations on arbitrary new formulas.

B The arithmetic interface and the inherited bridge

The reference theory T_{eq} is intuitionistic first-order equality logic over scalar iterator terms, with the two computation equations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and induction for literal equations under side assumptions independent of the induction variable. Its equality is ordinary equality. The reference iPRA has a symbol for each fixed primitive recursive definition, its defining equations, successor separation and injectivity, and quantifier-free induction over ordinary intuitionistic first-order logic.

The pinned development establishes $\mathcal{N} \simeq T_{\mathrm{eq}} \simeq$ iPRA [3]. The arithmetic part is retained unchanged here. Its substantive stages are:

1. Construct a predecessor in the scalar iterator language. Since the update binds only its previous state, this is a real construction; the counter variable is not silently made available to the update.
2. Derive equality reflection into a term and Boolean operations. Replace a quantifier-free formula by its characteristic equation, which transfers literal-equation induction to quantifier-free induction.
3. Construct pairing and projections, and compile a counter-aware primitive recursion by iterating a state that carries both the counter and the accumulated value.
4. Translate each fixed primitive recursive definition and each scalar iterator term in the opposite direction, proving their defining equations and both composites by equation induction.

The archive contains the corresponding complete written sources `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex` in the pinned reference subtree, together with the ordinary bridge and the native logical basis. These files are included to expose the mathematical dependency; the new report does not claim to have discovered these arithmetic constructions.

There is no quantified code for an arbitrary primitive recursive program and no total universal primitive recursive evaluator in the theory. A compiler acts externally on one finite definition at a time. This distinction matters: adjoining a purported total universal evaluator would change the foundational argument.

Both new translations between $\mathcal{R}$ and $\mathcal{N}$ fix every numerical term and literal equation. Composing them with the reference arithmetic translations therefore retains whole numerical domains and actual equality. The graph of each numerical interpretation composite is provably $x = y$. Formula composites are proved for all formulas, so the conclusion is bi-interpretability, not merely a correspondence of closed equations or an equiconsistency argument.

The precise existing Lean endpoint names are

```
native_iff_pra,  pra_iff_native,
native_roundTrip,  pra_native_roundTrip,
native_inconsistent_iff.
```

Their source was retrieved and authenticated at the pinned commit. Lean/Lake was not available in this run, so these existing files were inspected rather than rebuilt, and no new Lean theorem about $\mathcal{R}$ is claimed.

C Artifacts and reproducibility

The source archive contains the following main items.

Path	Purpose
`ipra_uniform_progress.tex`, `ipra_uniform_progress.pdf`	Self-contained LaTeX article and compiled report.
`report.tex`, `sections/`	Editable modular sources.
`code/orbit_checker.py`	Complete typed checker for the new theory.
`code/native_checker.py`, `code/witness_checker.py`	Matched standalone old-kernel controls.
`code/ancillary/`	Separate compact implementation improvements.
`tests/`, `evidence/`	Certificates, untrusted producer, reproduction, measurements, and exact outcomes.
`reference/`	Authenticated source files and the inherited arithmetic and logic bridge.
`build.py`, `CLAIMS.json`, `SHA256SUMS`	Article rebuild, exact scope, and package integrity.

From the extracted package directory:

```
python3 code/orbit_checker.py < tests/certificates/identity_at_three_orbit.json
python3 -B tests/reproduce.py
python3 build.py
```

The first command prints `ACCEPT`. The second regenerates the main certificates, performs the documented checks, and rewrites the evidence with the current Python and compression versions. The last assembles the standalone source and runs `pdflatex` enough times to settle references. Python execution uses only the standard library. PDF generation requires the standard TeX packages declared in the source.

The complete flattened source can also be compiled by itself:

```
pdflatex -interaction=nonstopmode -halt-on-error ipra_uniform_progress.tex
pdflatex -interaction=nonstopmode -halt-on-error ipra_uniform_progress.tex
```

The arithmetic and native/iPRA theorem sources are authenticated by their Git blob SHA-1 identifiers as well as SHA-256 content hashes. `reference/PROVENANCE.json` records their immutable paths and commit. `evidence/incoming_archives.json` records the six inspected ZIP blob identities. The source repository uses the MIT No Attribution license, included as `reference/LICENSE-ProveIt.txt`.

C.1 The seven proof-instruction branches

The following is the actual rule-synthesis function from the standalone typed checker. Its syntax validators, capture-avoiding substitution, scope operations, and file interface are also included in the measured module; they are not hidden dependencies.

```
def derive(proof, context=(), scope=0):
    require(type(proof) is list and proof)
    rule, *args = proof
    if rule == "hyp":
        index, = args
        require(type(index) is int and 0 <= index < len(context))
        return context[index]
    if rule == "zero":
        seed, body = args
        require(term(seed, scope) and term(body, scope + 1))
        return ["=", ["i", 0, seed, body], seed]
    if rule == "succ":
        counter, seed, body = args
        require(term(counter, scope) and term(seed, scope) and term(body, scope + 1))
        return ["=", ["i", ["s", counter], seed, body], instance(body, ["i", counter, seed,
    body])]
    if rule == "tr":
        matrix, major, minor = args
        require(formula(matrix, scope + 1) and matrix[0] == "=")
        equality = derive(major, context, scope)
        require(equality[0] == "=")
        require(derive(minor, context, scope) == instance(matrix, equality[1]))
        return instance(matrix, equality[2])
    if rule == "use":
        source, seed, major, minor = args
        require(term(source, scope) and term(seed, scope))
        closure = derive(major, context, scope)
        require(closure[0] == "q")
        matrix = closure[1]
        source = source if matrix[0] == "=" else 0
        require(derive(minor, context, scope) == instance(matrix, source))
        return instance(matrix, ["i", source, seed, ["s", ["v", 0]]])
    if rule == "elim":
        conclusion, major, continuation = args
        require(formula(conclusion, scope))
        paired = derive(major, context, scope)
        require(paired[0] == "d")
        local = (["q", paired[1]],) + tuple(lift(item) for item in context)
        require(derive(continuation, local, scope + 1) == lift(conclusion))
        return conclusion
    if rule == "intro":
        matrix, witness, premise = args
        require(formula(matrix, scope + 2) and term(witness, scope))
        body = substitute(matrix, 1, lift(witness))
        source = ["v", 0] if body[0] == "=" else 0
        local = (fresh_instance(body, source),) + tuple(lift(item) for item in context)
        require(derive(premise, local, scope + 1) == fresh_instance(body, ["s", ["v", 0]]))
        return ["d", matrix]
    raise ValueError("Unknown inference rule")
```

References

[1] Vladimir Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. Repository snapshot `a4e5e96c456303cf231eab2aeeb3291d8889833f`, inspected November 11, 2026. ConciseIPRA directory and README.

[2] ProveIt contributors. *Native scalar syntax, deduction, gates, and logical macros*. Native.lean, NativeGate.lean, and NativeLogic.lean, same snapshot.

[3] ProveIt contributors. *Native/iPRA bi-interpretability and its arithmetic foundations*. NativeBiInterpretation.lean and certified-zero reference proofs, same snapshot.

[4] ProveIt contributors. *One introduction rule*. NativeOneIntro.lean, same snapshot.

[5] ProveIt contributors. *A guarded core with a deterministic introduction rule*. Research/-guarded_core, same snapshot.

[6] ProveIt contributors. *Source-selected elimination and zero-source admissibility*. Research/-source_dispatch, same snapshot.

[7] ProveIt contributors. *A progressive binder presentation of iPRA*. Research/research/theory/progressive_binder.tex, same snapshot.

[8] ProveIt research intake. Six incoming iPRA archives, November 11, 2026. docs/incoming, same snapshot. The ZIP Git blob identities are recorded in the accompanying evidence.